

Projeto e Análise de Algoritmos

Projeto Recursivo de Algoritmos

Atílio G. Luiz

Primeiro Semestre de 2026

Projeto recursivo de algoritmos

Resolvendo um problema com recursão

1. **instâncias pequenas:** resolver diretamente
2. **instâncias grandes:**
 - ▶ construir uma instância menor do mesmo problema
 - ▶ encontrar e definir um **subproblema** adequado
 - ▶ resolver essa subinstância recursivamente
 - ▶ supor que sabemos resolver instâncias menores
 - ▶ encontrar uma solução para a instância original

Recursão e indução

Este processo é análogo a **demonstrações por indução**

- ▶ **caso base:** instâncias pequenas
- ▶ **caso geral:** instâncias grandes
 - ▶ a chamada recursiva corresponde à **hipótese de indução**
 - ▶ a construção da solução corresponde ao **passo da indução**

Algumas vantagens

- ▶ A **correção** do algoritmo vem do próprio processo indutivo
- ▶ A **complexidade** é expressa numa recorrência

Passos do projeto por indução

1. **Especificação: pré-condições e pós-condições.**

- ▶ Ainda mais importante nos recursivos, pois deve garantir as pré-condições nas subinstâncias, e depende do fato das pós-condições serem satisfeitas nas subsoluções

2. **Tamanho:** medida de tamanho da instância

3. **Entrada geral: considere uma instância grande e geral**

- ▶ Assuma que chamadas recursivas retornam subsoluções que atendem pós-condições, desde que você forneça subinstâncias menores (medida tamanho) que satisfaçam as pré-condições
- ▶ Obtenha e combine as subsoluções, para produzir a solução da instância original

4. Generalização do problema

- ▶ Pode ser necessário receber mais informação da entrada (pré-condições mais fortes)
 - ▶ Neste caso você tem que passar esta informação nas chamadas recursivas
- ▶ Pode ser necessário receber mais informação das subsoluções (pós-condições mais fortes)
 - ▶ Neste caso você tem que retornar também esta informação adicional
- ▶ Evite passar muita informação: deixe o problema o mais natural possível
- ▶ Aqui, as pré-condições e pós-condições cumprem o papel do invariante de laço

Passos do projeto por indução

5. Minimização do número de casos:

- ▶ Algoritmo deve funcionar para toda entrada que satisfaça as pré-condições. Pode ser necessário ter que tratar vários casos.
- ▶ Considere os casos do mais geral para o mais particular. Verifique se o caso particular já é atendido pelos casos mais gerais
- ▶ Ex.: Caso geral de árvore binária: nó com dois filhos. Caso particular: folha ou nó com apenas um dos filhos

6. Casos base:

- ▶ Resolva por força bruta quando a instância for suficientemente pequena

7. Tempo de execução:

- ▶ Obtenha a relação de recorrência e calcule a aproximação Θ usando alguma das técnicas vistas.

Subsequência consecutiva máxima

Subsequência consecutiva máxima (SCM)

Problema:

Dada uma sequência $X = x_1, x_2, \dots, x_n$ de números reais encontrar uma **subsequência consecutiva** $Y = x_i, x_{i+1}, \dots, x_j$, onde $1 \leq i, j \leq n$, cuja soma seja máxima.

Se todos os números na sequência forem negativos, então a subsequência consecutiva máxima é vazia, e o valor da subsequência vazia é 0.

Exemplos:

$X = [4, 2, -7, 3, 0, -2, 1, 5, -2]$ Resp: $Y = [3, 0, -2, 1, 5]$

$X = [-1, -2, 0]$ Resp: $Y = [0]$ ou $Y = []$

$X = [-3, -1]$ Resp: $Y = []$

Podemos facilmente projetar um algoritmo para este problema usando força bruta:

- ▶ Teste todos os pares de índices i e j com $1 \leq i \leq j \leq n$
- ▶ Existem $\binom{n}{2} + n$ destes pares, que é $\Theta(n^2)$.

Será que podemos fazer melhor que isso?

- ▶ Vamos tentar usar indução matemática.

Hipótese de Indução

Hipótese de indução

Sabemos calcular a SCM de sequências de comprimento $n - 1$.

Passo indutivo

- ▶ considere uma sequência $X = x_1, x_2, \dots, x_n$
- ▶ removendo x_n , obtemos uma sequência menor X'
- ▶ usando a hipótese de indução, obtemos uma SCM $Y' = x_i, x_{i+1}, \dots, x_j$ de X'

Há três casos a analisar

1. $Y' = \emptyset$: se $x_n \geq 0$, faça $Y = x_n$, senão faça $Y = \emptyset$
2. $j = n - 1$: se $x_n \geq 0$, faça $Y = Y' \parallel x_n$, senão faça $Y = Y'$
3. $j < n - 1$: consideramos dois subcasos
 - ▶ se Y' for uma SCM de X , então basta fazer $Y = Y'$
 - ▶ caso contrário, x_n faz parte de uma SCM de X e fazemos $Y = x_k, x_{k+1}, \dots, x_n$, para algum $k \leq n - 1$.
 - ▶ Exemplo: $[-20, 1, 2, -4, 4, -2, 50]$

- ▶ podemos executar os dois primeiros casos rapidamente
- ▶ mas para o último caso, a h.i. não fornece o valor de k
- ▶ mas, nesse caso, sabemos que uma SCM é

$$Y = x_k, x_{k+1}, \dots, x_{n-1}, x_n,$$

- ▶ ou seja, Y é um sufixo de X
- ▶ basta conhecer também o sufixo de maior soma de X'

Hipótese de indução reforçada

Hipótese de indução reforçada

Sabemos calcular a **SCM** e o sufixo de maior soma em sequências de comprimento $n - 1$.

Caso Base:

- ▶ consideramos a sequência vazia, de tamanho $n = 0$
- ▶ nesse caso, o sufixo de maior soma e a **SCM** são vazias. Logo, a soma nesses casos é igual a 0.

Entrada e saída do subproblema

Entrada (Pré-condições):

- ▶ um inteiro n
- ▶ Sequência de n números reais $X = x_1, x_2, \dots, x_n$

Saída (pós-condições):

- ▶ inteiros i, j, k tais que
 - ▶ $Y = x_i, x_{i+1}, \dots, x_j$ é uma SCM de X
 - ▶ tal que $Y = \emptyset$ se, e somente se, $i > j$
 - ▶ $S = x_k, x_{k+1}, \dots, x_n$ é um sufixo de soma máxima de X
 - ▶ tal que $S = \emptyset$ se, e somente se, $k > n$
- ▶ números reais não negativos $maxSeq, maxSuf$ tais que
 - ▶ a soma da sequência Y é $maxSeq$
 - ▶ a soma da sequência S é $maxSuf$

Algoritmo

```
SCM( $X, n$ ):  
1   se  $n = 0$   
2       então  $i = k = 1, j = \text{maxSeq} = \text{maxSuf} = 0$   
3   senão  
4        $i, j, k, \text{maxSeq}, \text{maxSuf} = \text{SCM}(X, n-1)$   
5        $\text{maxSuf} = \text{maxSuf} + x_n$   
6       se  $\text{maxSuf} < 0$   
7           então  $k = n + 1, \text{maxSuf} = 0$   
8       se  $\text{maxSuf} > \text{maxSeq}$   
9           então  $i = k, j = n, \text{maxSeq} = \text{maxSuf}$   
10  devolva  $i, j, k, \text{maxSeq}, \text{maxSuf}$ 
```

- ▶ **Entrada:** sequência $X = x_1, \dots, x_n$ e inteiro n .
- ▶ **Saída:** inteiros $i, j, k, \text{maxSeq}, \text{maxSuf}$ nesta ordem.

O tempo de execução $T(n)$ de SCM é

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 0; \\ T(n-1) + \Theta(1) & \text{se } n \geq 1. \end{cases}$$

A solução desta recorrência pelo método da árvore de recorrência nos dá:

$$\sum_0^n \Theta(1) = \Theta(1) \cdot \sum_0^n 1 = (n+1) \cdot \Theta(1) = \Theta(n).$$

Problema da celebridade

Definição

Em um conjunto S de n pessoas, uma **celebridade** é alguém que é conhecido por todas as pessoas de S mas que não conhece ninguém do grupo.

- ▶ queremos saber se existe uma celebridade em S
- ▶ em um grupo S , nem sempre existe uma celebridade
- ▶ mas só pode existir no máximo uma celebridade no grupo

O problema da celebridade

Formalizando o problema:

- ▶ queremos descrever o conhecimento entre n pessoas
- ▶ representamos isso com uma matriz binária $n \times n$
- ▶ $M[i, j] = 1$ se i conhece j e $M[i, j] = 0$ caso contrário.
- ▶ Não nos preocupamos com o valor de $M[i, i]$.

Problema

Dado um conjunto de n pessoas e a matriz associada M , encontrar uma celebridade no conjunto se existir.

Observações

- ▶ para uma celebridade k , vale $M[i, k] = 1$ para todo $i \neq k$
- ▶ analogamente, vale $M[k, i] = 0$ para todo $i \neq k$
- ▶ um algoritmo de força bruta verificaria se cada pessoa é celebridade, usando $n(n-1)$ operações

Argumento indutivo

Um argumento indutivo pode ser:

Hipótese de indução:

Sabemos encontrar uma celebridade em um conjunto de $n - 1$ pessoas ou decidir que não existe.

- ▶ se $n = 1$, a única pessoa no conjunto é uma celebridade
- ▶ se $n \geq 2$
 - ▶ represente o conjunto de pessoas como $S = \{1, 2, \dots, n\}$
 - ▶ podemos encontrar celebridade em $S' = \{1, 2, \dots, n - 1\}$
 - ▶ como isso ajuda na instância original?

Passo indutivo

Dois casos:

1. Existe celebridade k em S'
 - ▶ se $M[n, k] = 1$ e $M[k, n] = 0$, então k é celebridade em S
 - ▶ senão, n ainda poderia ser celebridade
 2. Não existe celebridade em S'
 - ▶ nesse caso, apenas n poderia ser celebridade
-
- ▶ pode ser necessário verificar se n é celebridade
 - ▶ temos que verificar se $M[n, j] = 0$ e $M[j, n] = 1$ para todo $j < n$
 - ▶ Desvantagem: também obtemos um algoritmo $O(n^2)$

Segunda tentativa

Seja $s \in S$. Suponha que s não é celebridade

- ▶ podemos fazer a chamada recursiva para $S' = S \setminus \{s\}$
- ▶ e não precisaríamos verificar se s é celebridade depois

Como encontrar alguém que **não** é celebridade?

- ▶ considere duas pessoas i e j
- ▶ se $M[i, j] = 1$, então i não é celebridade
- ▶ mas se $M[i, j] = 0$, então j não é celebridade

Vamos usar a mesma hipótese anterior

- ▶ i.e., consideramos o mesmo subproblema
- ▶ mas escolhemos a subinstância mais cuidadosamente

Segunda tentativa (cont.)

Hipótese de indução:

Sabemos encontrar uma celebridade em um conjunto S' de $n-1$ pessoas ou decidir que não existe.

Passo indutivo:

- ▶ Sabemos que a n -ésima pessoa que eliminamos não é uma celebridade.
- ▶ Assim, somente dois casos podem ocorrer agora:
 1. Não existe nenhuma celebridade em S' e, portanto, não existe celebridade em S .
 2. A celebridade está entre as $n-1$ pessoas de S' .
- ▶ O Caso 1 é trivial.
- ▶ Para resolver o Caso 2, basta perguntar se a n -ésima pessoa conhece a celebridade de S' e se a celebridade de S' não conhece a n -ésima pessoa.

Entrada e Saída do Algoritmo

- ▶ **Entrada (pré-condições):**

- ▶ conjunto $S = \{1, 2, \dots, n\}$ de inteiros.
- ▶ matriz binária quadrada M de dimensão $n \times n$.

- ▶ **Saída (pós-condições):**

- ▶ A saída é um inteiro i tal que:
 - ▶ $i = -1$ se não existe celebridade em S ;
 - ▶ $1 \leq i \leq n$ caso contrário.

Encontrando uma celebridade

Celebridade(S, M)

```
1  se  $|S| == 1$  então
2      seja  $k$  a única pessoa em  $S$ 
3  senão
4      sejam  $i, j$  duas pessoas em  $S$ 
5      se  $M[i, j] == 1$ 
6          então  $s = i$ 
7          senão  $s = j$ 
8       $S' = S \setminus \{s\}$ 
9       $k = \text{CELEBRIDADE}(S', M)$ 
10     se  $k == -1$  ou  $M[s, k] == 0$  ou  $M[k, s] == 1$ 
11         então  $k = -1$ 
12  devolva  $k$ 
```

Encontrando uma celebridade — Complexidade

Seja $T(n)$ o número de operações executadas

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1; \\ T(n-1) + \Theta(1) & \text{se } n > 1. \end{cases}$$

A solução da recorrência é

$$\sum_1^n \Theta(1) = n \cdot \Theta(1) = \Theta(n).$$

Divisão e conquista

Projeto de algoritmos por divisão e conquista

Relembre que um algoritmo de **divisão e conquista**

- ▶ divide uma instância do problema considerado em partes
- ▶ combina as várias soluções parciais

Algumas características

- ▶ normalmente criamos pelo menos duas subinstâncias
- ▶ elas podem ser muito menores que a instância original

Divisao-Conquista(x)

```
1  se a instância  $x$  é suficientemente pequena então
2    devolva SOLUCAO( $x$ )
3  senão
4    decomponha  $x$  em instâncias menores  $x_1, x_2, \dots, x_k$ 
5    para  $i$  de 1 até  $k$  faça
6       $y_i = \text{DIVISAO-CONQUISTA}(x_i)$ 
7    combine as soluções  $y_i$  para obter uma solução  $y$ 
8    devolva  $y$ 
```

Divisão e conquista

- ▶ Busca binária

Problema:

Dado um vetor de números ordenados A com n elementos e um número x , determinar alguma posição $i \in \{1, \dots, n\}$ tal que $A[i] = x$, ou retornar -1 se x não estiver no vetor.

- ▶ qual a melhor forma de decompor esse problema em um problema menor?
- ▶ vamos decompor o vetor em duas partes com cerca de metade dos elementos

Como representar subvetores?

Entrada (pré-condições):

- ▶ vetor ordenado $A[e..d]$ com $n = d - e + 1$ números
- ▶ índice e do elemento mais à esquerda do subvetor
- ▶ índice d do elemento mais à direita do subvetor
- ▶ número x

Saída (pós-condições):

- ▶ se x estiver no vetor, índice $i \in \{e, \dots, d\}$ tal que $A[i] = x$
- ▶ senão, retorna -1

Caso geral e Caso base

Qual o caso geral do algoritmo?

- ▶ vetor $A[e..d]$ tem pelo menos um elemento, ou seja $e \leq d$
- ▶ verificamos se o elemento no meio do vetor é x . Se for encontramos.
- ▶ caso contrário, dividimos o problema em 2 subproblemas de tamanho aproximadamente $n/2$ e excluimos um dos subproblemas.
- ▶ por hipótese, se x estiver na metade escolhida, o seu índice será retornado; senão, será retornado -1 .

Qual o caso base do algoritmo?

- ▶ vetor $A[e..d]$ é vazio, ou seja $e > d$.
- ▶ neste caso, retornamos -1

Algoritmo

Busca-Binaria(A, e, d, x)

1 se $e > d$ então

2 devolva -1

3 senão

4 $i = \lfloor \frac{e+d}{2} \rfloor$

5 se $x == A[i]$ então

6 devolva i

5 se $x < A[i]$ então

6 devolva **BUSCA-BINARIA**($A, e, i - 1, x$)

7 senão

8 devolva **BUSCA-BINARIA**($A, i + 1, d, x$)

O tempo de execução é:

$$T(n) = \begin{cases} 1 & \text{se } n = 0; \\ T(\lceil \frac{n}{2} \rceil) + \Theta(1) & \text{se } n \geq 1. \end{cases}$$

Resolvendo-se a recorrência pelo Teorema Mestre, temos

$$T(n) = \Theta(\log n)$$

Correção da BUSCA-BINARIA

Como demonstrar a correção de um algoritmo recursivo?

- ▶ podemos usar indução diretamente
- ▶ precisamos verificar a corretude das demais sub-rotinas, se existirem

Correção da BUSCA-BINARIA

- ▶ queremos mostrar que BUSCA-BINARIA acha o elemento x se ele estiver presente no vetor $A[e..d]$
- ▶ basta usar indução no tamanho do vetor, ou seja $n = d - e + 1$

Correção da BUSCA-BINARIA

Base da indução:

- ▶ quando $e > d$, temos que $n = 0$
- ▶ neste caso, o vetor é vazio e, portanto, x não está presente.

Caso geral:

- ▶ considere um vetor A crescente de tamanho n
- ▶ hipótese indutiva: suponha que **BUSCA-BINARIA** acha x se ele estiver presente em vetores crescentes menores que n .
- ▶ ao comparar x com o elemento do meio de A , o $A[i]$, determinamos se $x = A[i]$ ou se $x < A[i]$ ou se $x > A[i]$.
- ▶ se $x = A[i]$, então o índice i é retornado
- ▶ caso contrário, **BUSCA-BINARIA** é invocada para a metade do vetor onde x provavelmente está, e, pela hipótese de indução, essa chamada determina se x está presente ou não no vetor.

Divisão e conquista

- ▶ Algoritmo de Strassen

Multiplicação de Matrizes

Problema:

Dadas duas matrizes quadradas $A = (a_{ij})$ e $B = (b_{ij})$ de dimensões $n \times n$, calcular o produto $C = A \cdot B$.

- **Obs.:** Na matriz-produto $C = A \cdot B$, a entrada c_{ij} é definida matematicamente pelo produto escalar da i -ésima linha de A com a j -ésima coluna de B .

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

Exemplo ($n = 2$)

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad \text{e} \quad B = \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

$$A \cdot B = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

Algoritmo simples (iterativo)

Square-Matrix-Multiply(A, B, n)

Entrada: matrizes quadradas A, B de ordem n

Saída: $C = A \cdot B$

```
1  seja  $C$  uma nova matriz  $n \times n$ 
2  para  $i = 1$  até  $n$ 
3      para  $j = 1$  até  $n$ 
4           $c_{ij} = 0$ 
5          para  $k = 1$  até  $n$ 
6               $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$ 
7  devolva  $C$ 
```

Complexidade: $\Theta(n^3)$.

É possível fazer melhor?

Uma abordagem usando divisão e conquista

- ▶ Sejam A e B matrizes quadradas de ordem $n \times n$.
- ▶ Por conveniência, vamos assumir que n é uma potência de 2.
- ▶ Como quebrar o problema em subproblemas menores e como combinar as soluções desses subproblemas em uma solução do problema original?

Uma abordagem usando divisão e conquista

A e B podem ser subdivididas em 4 submatrizes de ordem $\frac{n}{2} \times \frac{n}{2}$:

$$A = \left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right] \quad \text{e} \quad B = \left[\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array} \right]$$

Podemos reescrever $C = A \cdot B$ como:

$$\left[\begin{array}{c|c} C_{11} & C_{12} \\ \hline C_{21} & C_{22} \end{array} \right] = \left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right] \cdot \left[\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array} \right]$$

ou, equivalentemente:

$$\left[\begin{array}{c|c} C_{11} & C_{12} \\ \hline C_{21} & C_{22} \end{array} \right] = \left[\begin{array}{c|c} A_{11} \cdot B_{11} + A_{12} \cdot B_{21} & A_{11} \cdot B_{12} + A_{12} \cdot B_{22} \\ \hline A_{21} \cdot B_{11} + A_{22} \cdot B_{21} & A_{21} \cdot B_{12} + A_{22} \cdot B_{22} \end{array} \right] \quad (1)$$

Essa decomposição se traduz naturalmente em uma algoritmo recursivo.

Algoritmo simples (divisão-e-conquista)

SMM-Rec(A, B)

```
1   $n = A.rows$ 
2  seja  $C$  uma nova matrix  $n \times n$ 
3  se  $n == 1$ 
4       $c_{11} = a_{11} \cdot b_{11}$ 
5  senão
6      particione  $A$ ,  $B$  e  $C$  como na Equação (1)
7       $C_{11} = \text{SMM-REC}(A_{11}, B_{11}) + \text{SMM-REC}(A_{12}, B_{21})$ 
8       $C_{12} = \text{SMM-REC}(A_{11}, B_{12}) + \text{SMM-REC}(A_{12}, B_{22})$ 
9       $C_{21} = \text{SMM-REC}(A_{21}, B_{11}) + \text{SMM-REC}(A_{22}, B_{21})$ 
10      $C_{22} = \text{SMM-REC}(A_{21}, B_{12}) + \text{SMM-REC}(A_{22}, B_{22})$ 
11  devolva  $C$ 
```

Qual a complexidade da linha 06?

Como faríamos para implementar a linha 06 com complexidade $\Theta(1)$?

Complexidade de SMM-REC

A complexidade do algoritmo SMM-REC pode ser expressa pela seguinte recorrência:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1; \\ 8T(n/2) + \Theta(n^2) & \text{se } n > 1. \end{cases} \quad (2)$$

- ▶ Aplicando o Teorema Mestre na recorrência (2) acima, obtemos que $T(n) = \Theta(n^3)$.
- ▶ Assim, essa abordagem simples de divisão e conquista não consegue ser mais rápida que o algoritmo iterativo simples SQUARE-MATRIX-MULTIPLY.

Método de Strassen (1969)



Volker Strassen

Ideia: Ao invés de realizar 8 multiplicações recursivas de matrizes de dimensões $\frac{n}{2} \times \frac{n}{2}$, é possível realizar apenas 7 dessas multiplicações.

O custo dessa economia é que teremos que realizar diversas adições e subtrações de matrizes de ordem $\frac{n}{2} \times \frac{n}{2}$. Porém será um número constante delas (18 somas e subtrações de matrizes)

Método de Strassen (1969)

Dadas A e B subdivididas em 4 submatrizes de ordem $\frac{n}{2} \times \frac{n}{2}$:

$$A = \left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right] \quad \text{e} \quad B = \left[\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array} \right]$$

Strassen formulou as 7 multiplicações de matrizes a seguir:

$$P_1 = A_{11} \cdot (B_{12} - B_{22})$$

$$P_2 = (A_{11} + A_{12}) \cdot B_{22}$$

$$P_3 = (A_{21} + A_{22}) \cdot B_{11}$$

$$P_4 = A_{22} \cdot (B_{21} - B_{11})$$

$$P_5 = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$$

$$P_6 = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$$

$$P_7 = (A_{11} - A_{21}) \cdot (B_{11} + B_{12})$$

Amazing equation:

$$A \cdot B = \left[\begin{array}{c|c} A_{11} \cdot B_{11} + A_{12} \cdot B_{21} & A_{11} \cdot B_{12} + A_{12} \cdot B_{22} \\ \hline A_{21} \cdot B_{11} + A_{22} \cdot B_{21} & A_{21} \cdot B_{12} + A_{22} \cdot B_{22} \end{array} \right]$$
$$= \left[\begin{array}{c|c} \frac{P_5 + P_4 - P_2 + P_6}{P_3 + P_4} & \frac{P_1 + P_2}{P_1 + P_5 - P_3 - P_7} \end{array} \right]$$

Portanto, é possível modificar o algoritmo de divisão e conquista SMM-REC a fim de que ele realize apenas 7 chamadas recursivas!

Algoritmo de Strassen

Descrição em alto nível:

Strassen(A, B)

1 $n = A.rows$

2 seja C uma nova matriz $n \times n$

3 se $n == 1$

4 $c_{11} = a_{11} \cdot b_{11}$

5 senão

6 particione A , B e C como na Equação (1)

7 compute, recursivamente, as sete matrizes $P_1, P_2, \dots, P_7$

8 $C_{11} = P_5 + P_4 - P_2 + P_6$

9 $C_{12} = P_1 + P_2$

0 $C_{21} = P_3 + P_4$

1 $C_{22} = P_1 + P_5 - P_3 - P_7$

2 devolva C

Complexidade de STRASSEN

A complexidade do algoritmo STRASSEN pode ser expressa pela seguinte recorrência:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1; \\ 7T(n/2) + \Theta(n^2) & \text{se } n > 1. \end{cases} \quad (3)$$

- ▶ Aplicando o Teorema Mestre na recorrência (3) acima, obtemos que $T(n) = \Theta(n^{\log_2 7}) = \Theta(n^{2.8})$, ou seja $T(n) = o(n^3)$.